

1 . Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

```
import gensim.downloader as api

# Load model
model = api.load("glove-wiki-gigaword-50")

# Perform vector arithmetic
result = model.most_similar(positive=['king', 'woman'], negative=['man'])

# Print result
print(result)

[('queen', 0.8523604273796082), ('throne', 0.7664334177970886), ('prince',
0.759214460849762), ('daughter', 0.7473882436752319), ('elizabeth',
0.7460220456123352), ('princess', 0.7424570322036743), ('kingdom',
0.7337412238121033), ('monarch', 0.721449077129364), ('eldest',
0.7184862494468689), ('widow', 0.7099431157112122)]

import gensim.downloader as api

# Load model
model = api.load("word2vec-google-news-300")

# Get vectors
v1 = model['king']
v2 = model['man']
v3 = model['woman']

# Perform arithmetic: king - man + woman
result_vector = v1 - v2 + v3

# Find most similar word
result = model.similar_by_vector(result_vector, topn=1)

# Print result
print(result[0])

('king', 0.8449392318725586)
```

2 Use Dimensionality Reduction (e.g., PCA Or T-SNE) To Visualize Word Embeddings For Q 1. Select 10 Words From A Specific Domain (e.g., Sports, Technology) And Visualize Their Embeddings. Analyze Clusters And Relationships. Generate Contextually Rich Outputs Using Embeddings. Write A Program To Generate 5 Semantically Similar Words For A Given Input.

```
import gensim.downloader as api
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.manifold import TSNE
```

```
import numpy as np
```

```
model = api.load("glove-wiki-gigaword-50")
```

```
words = ['king','queen','prince','princess','throne',  
         'kingdom','monarch','royal','crown','empire']
```

```
vec = np.array([model[w] for w in words])
```

```
for name, reducer in [("PCA", PCA(2)), ("t-SNE", TSNE(2, perplexity=5))]:
```

```
    res = reducer.fit_transform(vec)
```

```
    plt.figure()
```

```
    for i, w in enumerate(words):
```

```
        plt.scatter(res[i,0], res[i,1])
```

```
        plt.text(res[i,0], res[i,1], w)
```

```
    plt.title(name)
```

```
    plt.show()
```

```
print(model.most_similar('king', topn=5))
```

3 Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

```
from gensim.models import Word2Vec
```

```
from gensim.utils import simple_preprocess
```

```
# Small domain-specific corpus
```

```
sentences = [
```

```
    "legal contract agreement court law",
```

```
    "court judge legal case law",
```

```
    "medical doctor patient hospital treatment",
```

```
    "medical report patient diagnosis doctor"
```

```
]
```

```
# Tokenize
```

```
data = [simple_preprocess(s) for s in sentences]
```

```
# Train model
```

```
model = Word2Vec(data, vector_size=50, min_count=1)
```

```
# Find similar words
```

```
print(model.wv.most_similar("legal", topn=5))
```

4 Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

```
import gensim.downloader as api
```

```
from transformers import pipeline
```

```
# Load models
```

```
word_vectors = api.load("glove-wiki-gigaword-50")
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
# Original prompt
```

```
prompt = "Who is king"
```

```
# Get similar word
```

```
similar_word = word_vectors.most_similar("king", topn=1)[0][0]
```

```
# Create enriched prompt
```

```
enriched_prompt = prompt.replace("king", similar_word)
```

```
print("Original Prompt:", prompt)
```

```
print("Enriched Prompt:", enriched_prompt)
```

```
# Generate responses
```

```
original_output = generator(prompt, max_length=50)[0]['generated_text']
```

```
enriched_output = generator(enriched_prompt, max_length=50)[0]['generated_text']
```

```
# Print outputs
```

```
print("\nOriginal Response:\n", original_output)
```

```
print("\nEnriched Response:\n", enriched_output)
```

```
# Simple comparison

print("\nLength Comparison:")

print(len(original_output), "vs", len(enriched_output))
```

5 Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

```
import gensim.downloader as api
```

```
# Load model

model = api.load("glove-wiki-gigaword-50")
```

```
# Input seed word

seed = input("Enter a word: ")
```

```
# Get similar words

similar = [w[0] for w in model.most_similar(seed, topn=5)]
```

```
# Create paragraph

paragraph = f"""

The {seed} was known for its connection with {similar[0]} and {similar[1]}.

People often related it to {similar[2]} and {similar[3]}.

In many stories, {similar[4]} played an important role alongside the {seed}.

"""
```

```
print("\nGenerated Paragraph:\n")

print(paragraph)
```

6 Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real- world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

```
from transformers import pipeline

# Load sentiment model
analyzer = pipeline("sentiment-analysis")

# Sample inputs (real-world: customer reviews)
reviews = [
    "The product is amazing! I love it.",
    "I'm very disappointed. The service was terrible.",
    "It was an average experience.",
    "Absolutely fantastic quality!",
    "Not great, but okay."
]

# Analyze sentiment
for r in reviews:
    result = analyzer(r)[0]
    print(f'{r} → {result["label"]} ({result["score"]:.2f})')
```

7 Summarize long texts using a pre-trained summarization model using Hugging Face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

text = ""
```

Summarize this:

Artificial Intelligence (AI) is a rapidly evolving field focused on creating intelligent machines.

It has impacted industries like healthcare, finance, and education. AI applications include chatbots,

self-driving cars, and recommendation systems. However, it also raises ethical concerns like bias

and job displacement.

```
"""
```

```
output = generator(text, max_length=100, num_return_sequences=1)
```

```
print(output[0]['generated_text'])
```

```
from transformers import pipeline
```

```
summarizer = pipeline("text2text-generation", model="facebook/bart-large-cnn")
```

```
text = """
```

Artificial Intelligence is transforming industries like healthcare and finance.

It enables machines to learn and make decisions. However, it also raises ethical concerns.

```
"""
```

```
summary = summarizer("summarize: " + text, max_length=50)
```

```
print(summary[0]['generated_text'])
```

9 Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder

**of the Institution. When it was founded. The current branches in the institution.
How many employees are working in it. A brief 4-line summary of the institution.**

```
from pydantic import BaseModel
```

```
import wikipediaapi
```

```
# Define schema
```

```
class InstitutionDetails(BaseModel):
```

```
    founder: str | None
```

```
    founded: str | None
```

```
    summary: str | None
```

```
# Fetch data
```

```
def get_details(name):
```

```
    wiki = wikipediaapi.Wikipedia(
```

```
        user_agent="SkandaNLPPProject/1.0 (skanda@email.com)",
```

```
        language="en"
```

```
    )
```

```
    page = wiki.page(name)
```

```
    if not page.exists():
```

```
        print("Page not found")
```

```
        return None
```

```
    text = page.text.lower()
```

```
# Simple extraction (basic but works for lab)
```

```
    founder = "founder" if "founder" in text else None
```

```
    founded = "founded" if "founded" in text else None
```



```
return InstitutionDetails(  
    founder=founder,  
    founded=founded,  
    summary=page.summary[:300]  
)
```

```
# Main
```

```
name = input("Enter institution: ")
```

```
data = get_details(name)
```

```
if data:
```

```
    print("\nDetails:\n", data)
```